DAppGenius Documentation

Overview

DAppGenius is a cutting-edge decentralized application designed to streamline digital asset management, user interactions, and organizational operations. Built with a robust tech stack, it ensures scalability, security, and efficiency.

📌 Tech Stack

Component	Technology	Purpose
Core Language	Node.js + TypeScript	Backend development
HTTP Requests	Express.js	API routing and middleware handling
Validation Middleware	Celebrate	Schema validation for requests
Database Manager	TypeORM	ORM for PostgreSQL
Database	PostgreSQL	Primary relational database
In-memory Data Store	Redis	Caching and session management
SMS Communication	Twilio	Sending verification codes via SMS
Email Server	Amazon SES	Sending transactional emails
Token Management	JSON Web Tokens (JWT)	Secure information exchange and authentication

---

📌 Development Principles

- **SOLID Principles**: Ensures that the software is easy to maintain and extend over time.
  - SOLID: The First 5 Principles of Object-Oriented Design

📌 Redis Usage

Redis is used in the project as an in-memory data store to enhance performance and scalability. Key functionalities include:

- **Caching**: Frequently accessed data is cached to reduce database load and improve speed.
- **Session Management**: Stores user session data for quick access across distributed systems.
- **Real-time Data Processing**: Supports live updates, chat systems, and analytics.
- **Pub/Sub Messaging**: Enables efficient inter-service communication.
- **Node Voting Coordination**: Helps in distributed decision-making.

📌 Two-Factor Authentication (2FA) with Twilio

To enhance security, the project implements 2FA using Twilio's API:

1. **User Login Attempt** → Generates a verification code.
2. **Twilio SMS Service** → Sends the code to the user's phone.
3. **User Inputs Code** → Verifies and grants access.

For more information on how to generate verification codes using Twilio, visit the Twilio Documentation.

📌 Transaction Token Logic

In the DAppGenius system, each transaction is identified using a `sync_id`, which serves as a unique identifier for tracking and managing transactions. Here's how it works:

- **Generation**: The `sync_id` is generated at the start of a transaction process. It should be unique and securely generated to ensure it accurately identifies a transaction. Typically, a UUID is used for this purpose:

  ```
  const { v4: uuidv4 } = require('uuid');
  const sync_id = uuidv4();
  ```

- **Usage**:
  - **Transaction Creation**: The `sync_id` is used when creating transaction records in the database, linking related transactions (e.g., debit and credit entries).
  - **Audit Logging**: It is logged in audit records to track the transaction's lifecycle and actions taken.
  - **Memory Storage**: Used as part of the key in Redis to store and retrieve transaction-related votes, facilitating node coordination in distributed systems.
- **Security Considerations**: While `sync_id` is crucial for transaction tracking, it should not be used for authentication or authorization purposes.

📌 Security and Cryptography with JWT

JSON Web Tokens (JWT) are used in the project to securely transmit information between parties as a JSON object. They are commonly used for authentication and information exchange. Here's how JWT works in the context of this project:

How JWT Works

1. **Token Generation**: When a user logs in, a JWT is generated using a secret key. This token contains encoded information such as user ID and expiration time.

```
const jwt = require('jsonwebtoken');
const token = jwt.sign({ userId: user.id }, process.env.APP_SECRET,
{ expiresIn: '1h' });
```

2. **Token Structure**: A JWT consists of three parts: Header, Payload, and Signature.
   - **Header**: Contains metadata about the token, such as the type of token and the hashing algorithm used.
   - **Payload**: Contains the claims, which are statements about an entity (typically, the user) and additional data.
   - **Signature**: Used to verify that the sender of the JWT is who it says it is and to ensure that the message wasn't changed along the way.
3. **Token Verification**: When a user makes a request to a protected route, the server verifies the token using the secret key.

```
jwt.verify(token, process.env.APP_SECRET, (err, decoded) => {
  if (err) {
    // Handle error
  } else {
    // Proceed with the request
  }
});
```

4. **Security Considerations**:
   - Always keep your secret key secure and never expose it in your codebase.
   - Set an appropriate expiration time for tokens to enhance security.
   - Use HTTPS to protect tokens in transit.

Using jwt.io

- **jwt.io** is a helpful tool for decoding, verifying, and generating JWTs. You can paste your JWT into the tool to see its contents and verify its signature. For more information, visit jwt.io.

📌 Environment Configuration (.env.example)

The `.env.example` file provides a template for setting up environment variables required by the application. These variables are crucial for configuring the application to run in different environments (development, testing, production).

Setting Up for Local Development

1. **Copy the Template**: Duplicate the `.env.example` file and rename it to `.env` in the root directory of your project.

```
cp .env.example .env
```

2. **Fill in the Required Values**: Open the `.env` file and provide the necessary values for each variable. Here's a breakdown of some key variables:
   - **NODE_NUMBER**: The identifier for the node in a distributed setup.
   - **NODE_NAME**: The name of the node.
   - **APP_SECRET**: A secret key used for application security.
   - **REDIS_HOST** and **REDIS_PORT**: Host and port for the Redis server.
   - **TWILIO_ACCOUNT_SID** and **TWILIO_AUTH_TOKEN**: Credentials for Twilio services.
   - **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY**: Credentials for accessing AWS services.
3. **Ensure All Variables Are Set**: Make sure all necessary variables are filled out to avoid runtime errors.

📌 Folder Structure

```
📁 src
├ 📁 @types       # TypeScript custom types
├ 📁 config       # Configuration files (env, API keys)
├ 📁 modules
│ ├ 📁 dtos       # Data Transfer Objects (DTOs)
│ ├ 📁 infra      # Infrastructure layer
│ │ ├ 📁 http
│ │ │ ├ controllers  # Handles HTTP requests
│ │ │ ├ middlewares  # Request interceptors
│ │ │ └ routes       # API routes
│ │ ├ 📁 typeorm
│ │ │ ├ entities     # Database models
│ │ │ └ repositories # Data access logic
│ ├ 📁 services    # Business logic
│ ├ 📁 repositories # Interfaces for data sources
│ ├ 📁 shared
│ │ ├ container    # Dependency injection
│ │ ├ providers    # External service integrations
│ │ ├ errors       # Error handling
│ │ ├ infra
│ │ │ ├ http       # HTTP infrastructure
│ │ │ ├ routes     # API route definitions
│ │ │ └ Server.ts  # Server startup
│ │ ├ views        # Email templates
```

📌 Modules Overview

User Management

- Handles user authentication, profiles, and security features.

Digital Assets

- Manages asset creation, transactions, and blockchain integration.

Wallet

- Supports balance tracking, payments, and transaction history.

Groups

- Enables group-based permissions and collaborations.

Data Forms

- Dynamic form generation and user input collection.

Instant Messaging

- Real-time messaging with text, voice, and video support.

Organizations

- Manages company structures and role-based access.

📌 Features

- ✅ User Management
- ✅ Digital Asset Handling
- ✅ Wallet Transactions
- ✅ Group Management
- ✅ Dynamic Data Forms
- ✅ Real-time Messaging
- ✅ Organization Management

📌 Architecture Diagram

```
Client Layer  (React)
    ↓
Server Layer  (Node.js + Express)
    ↓
Database Layer (PostgreSQL + Redis)
    ↓
External Services (Twilio, Amazon SES)
```

📌 Running the Project

```
# Clone repository
git clone https://github.com/yourusername/yourproject.git
cd yourproject

# Install dependencies
npm install  # or yarn install

# Setup environment variables
cp .env.example .env

# Run database migrations
npm run typeorm migration:run  # or yarn typeorm migration:run

# Start the server
npm start  # or yarn start
```

📌 Docker Setup

```
# Build Docker image
docker build -t dappgenius .

# Run container
docker run -p 3000:3000 dappgenius
```

📌 Deployment on Digital Ocean

1. **Create a Droplet** on Digital Ocean.
2. **SSH into the server** and install necessary software.
3. **Clone the repository** and run the Docker setup.
4. **Configure domain & SSL** using Let's Encrypt.

📌 Common Issues and Solutions

Issue	Solution
Redis connection error	Ensure Redis is running and `.env` is configured correctly.
Twilio SMS not sending	Verify Twilio credentials and phone number formats.

📌 Testing

```
npm test  # or yarn test
```

📌 Contributing

We welcome contributions! Follow these steps:

```
# Fork the repository
git checkout -b feature/YourFeature
git commit -m "Add some feature"
git push origin feature/YourFeature
```

Then open a **Pull Request** on GitHub.

📌 Support

<<<<<<< HEAD
For any questions, email:
📧 **leo@mindwebsolutions.com**

Support

For support

3072ca301a677d3c8bd829bb27e41f9ca970411b